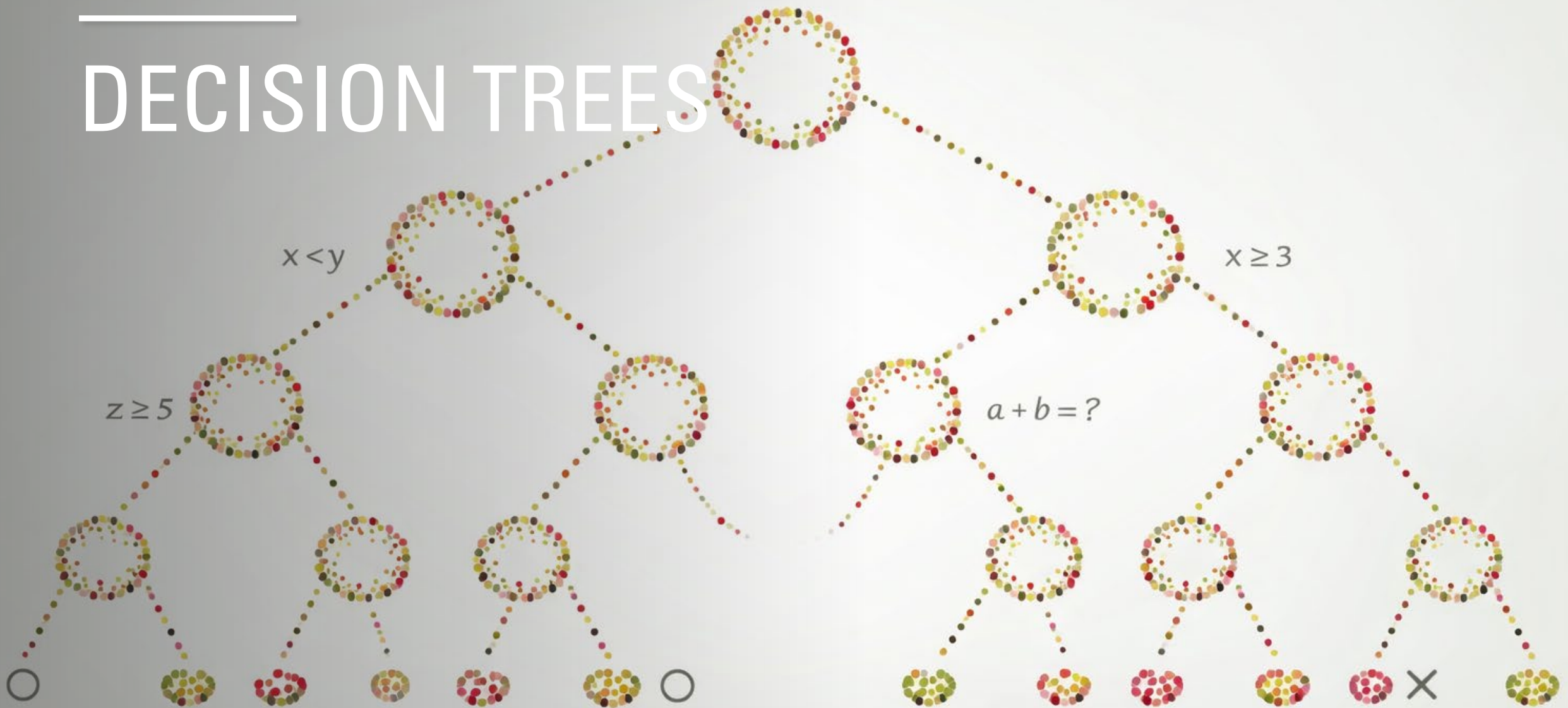
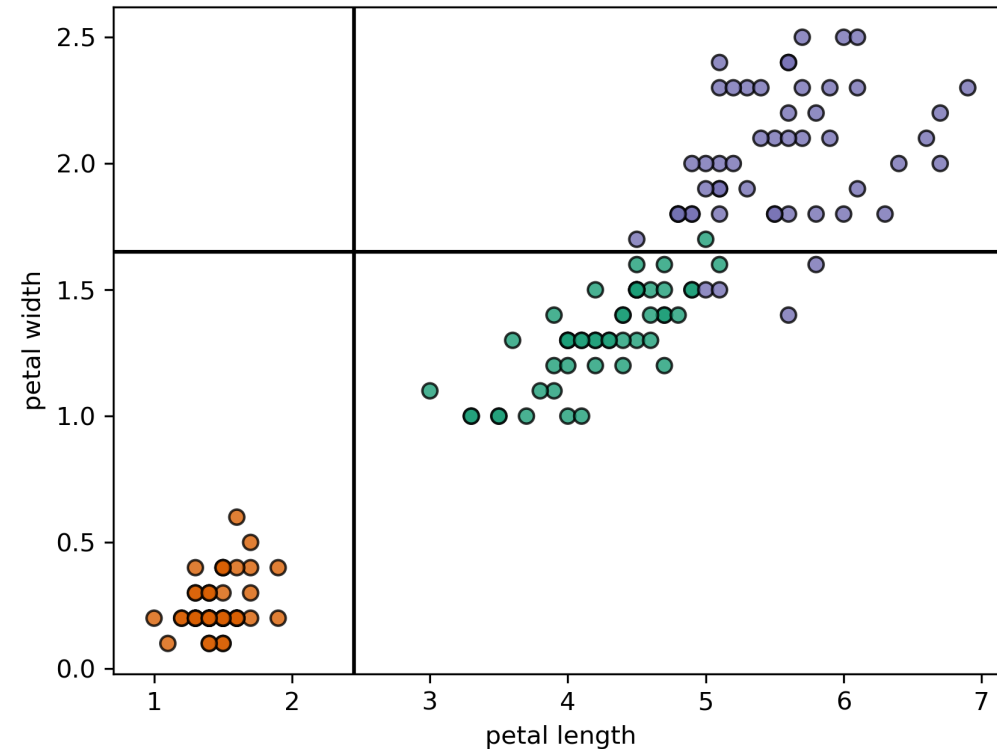
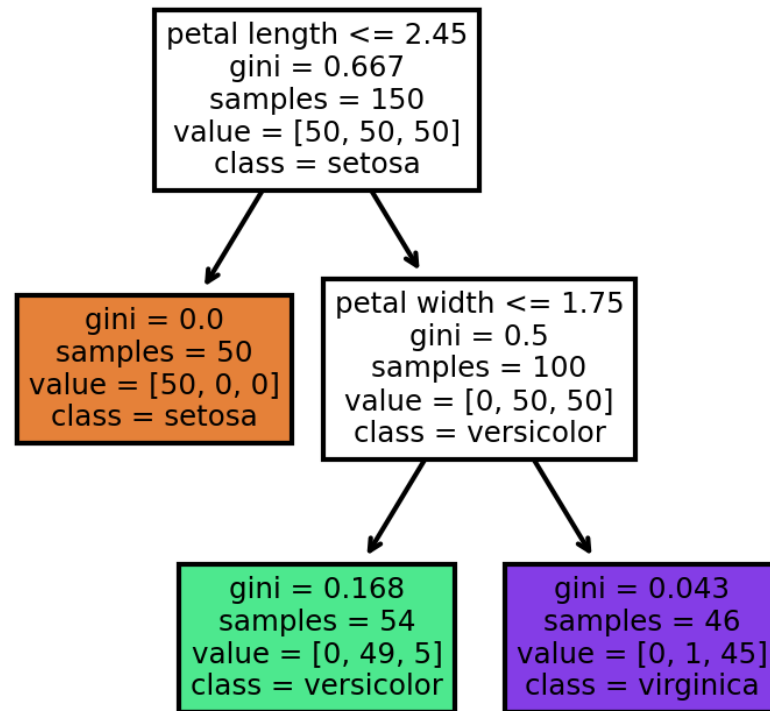


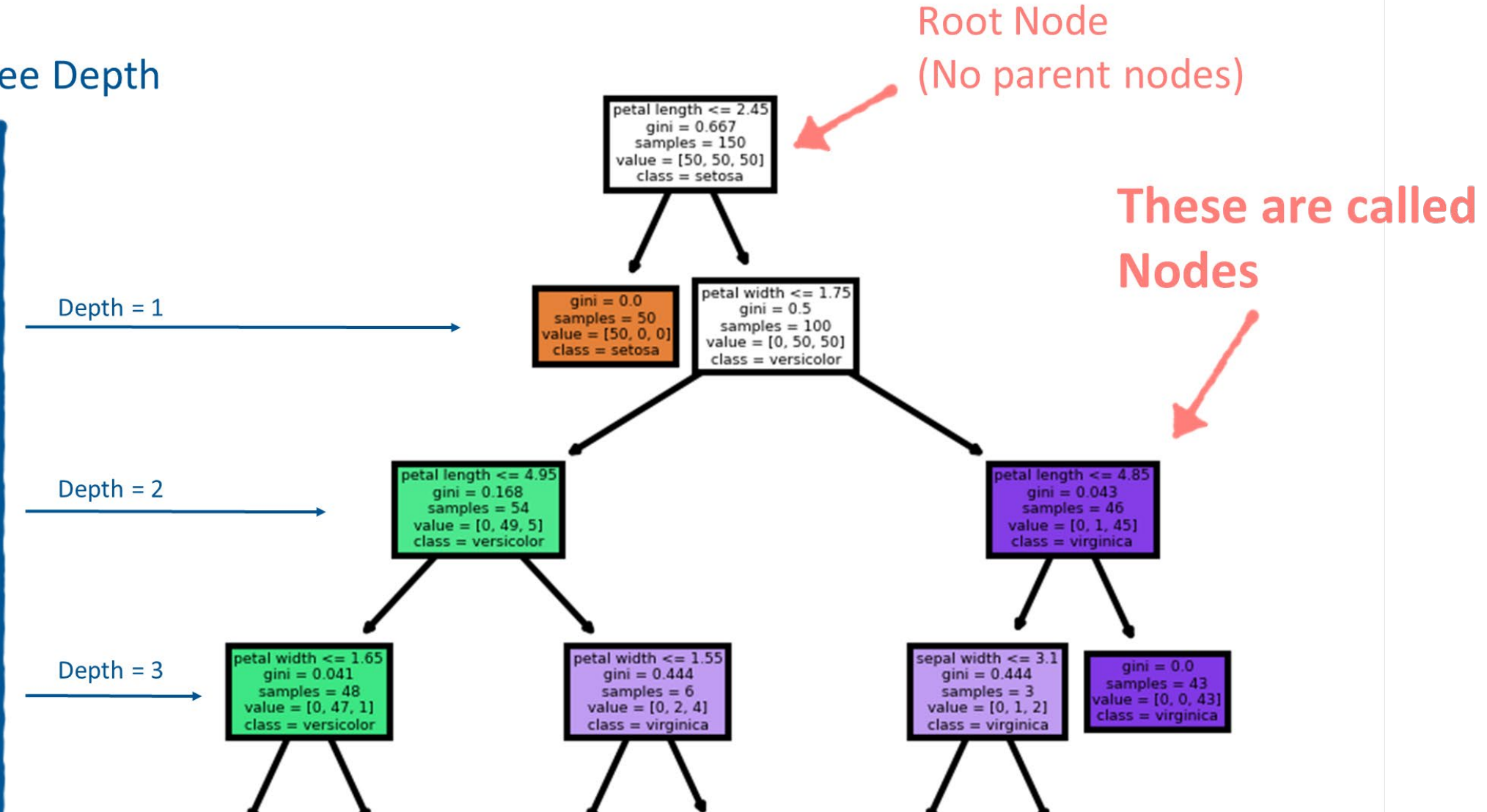
DECISION TREES



EXAMPLE DECISION TREE



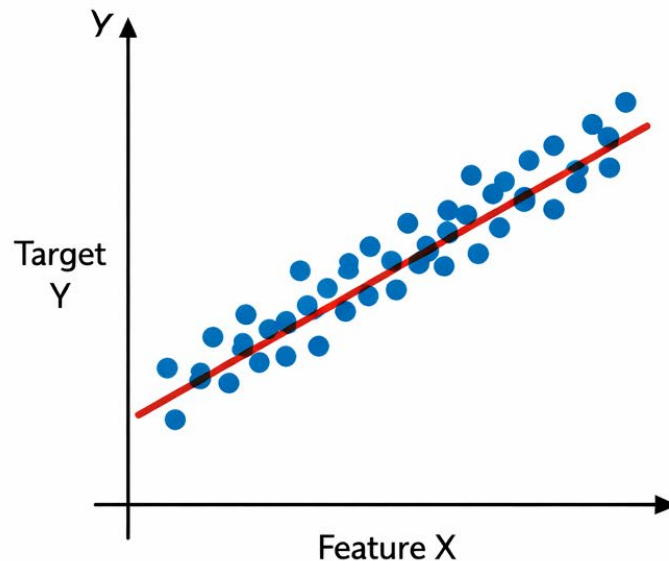
Tree Depth



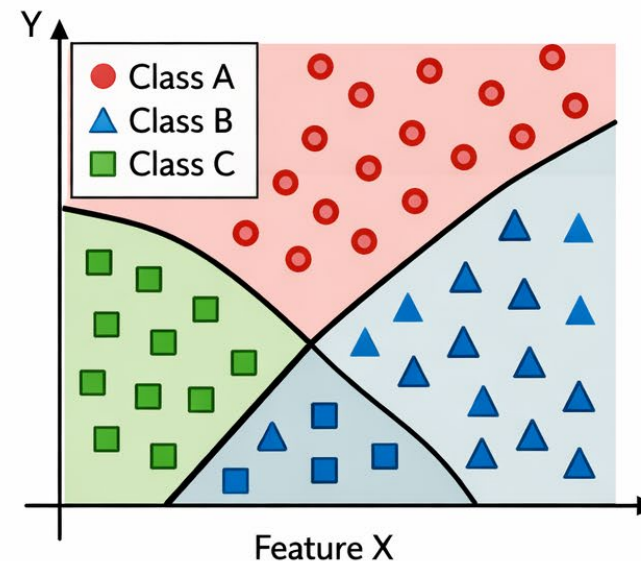
HOW TO FIT?

- Binary decision search algorithm.
- Partition (split) data to optimize some objective.
- What might that objective be?

Regression Problem



Classification Problem



DECISION TREE ALGORITHMS

- CART is one of the most popular decision tree algorithms
 - Binary splitting - trees are constructed by splitting the data into two subsets using the feature and threshold that results in the “best” split
 - Best split - the best split is the split that produces the most homogeneous groups
 - Recursive splitting - the process of splitting is applied recursively to each derived subset
 - Pruning - trees are pruned to obtain a smaller tree to help prevent overfitting
-

NODE IMPURITY MEASURES (CLASSIFICATION)

- Define p_{ik} as the proportion of instances in the i^{th} node that are from class k
- Gini index

$$G_i = 1 - \sum_{k=1}^K p_{ik}^2 = \sum_{k=1}^K p_{ik}(1 - p_{ik})$$

- Cross Entropy

$$H_i = - \sum_{\substack{k=1 \\ \text{if } p_{ik} \neq 0}}^K p_{ik} \log_2(p_{ik})$$

GROWING THE TREE (FITTING)

- Classification and Regression Tree (CART) algorithm:
 - Split the training data into two subsets using a single feature, k , and a threshold, t_k
 - Search for the (features, threshold) or (k, t_k) pair that minimizes the cost function:

CLASSIFICATION $\frac{n_{left}}{n} G_{left} + \frac{n_{right}}{n} G_{right}$ Where G is a measure of node impurity

REGRESSION $\frac{n_{left}}{n} MSE_{left} + \frac{n_{right}}{n} MSE_{right}$

NODE IMPURITY MEASURES (CLASSIFICATION)

- Gini index : $G_i = 1 - \sum_{k=1}^K p_{ik}^2$

- For the “green” node

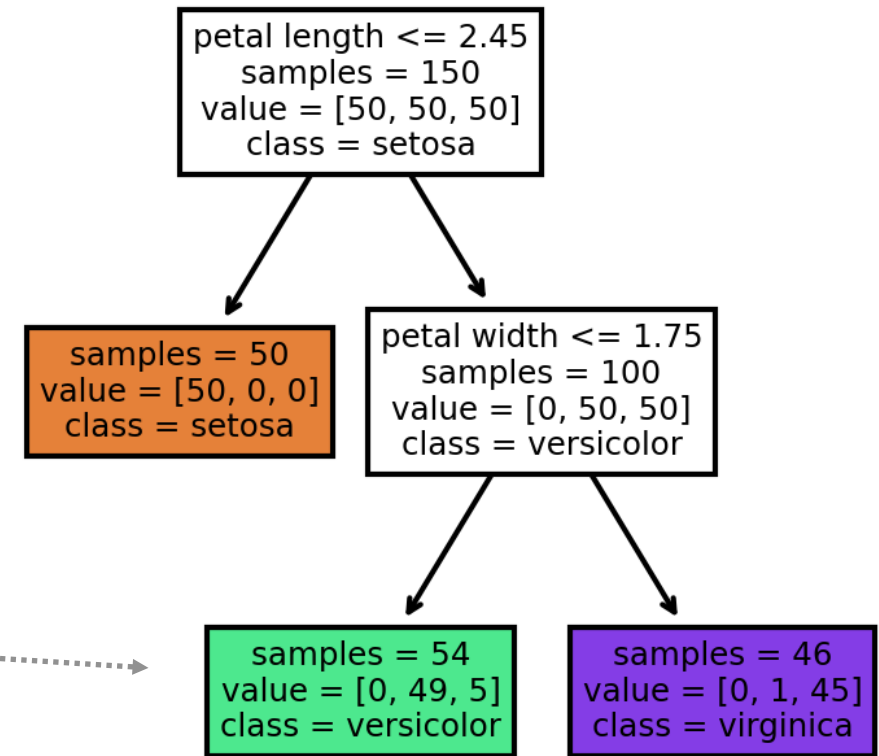
$$p_{i1} = (0/54); p_{i2} = (49/54); p_{i3} = (5/54)$$

$$G_i = 1 - ((0/54)^2 + (49/54)^2 + (5/54)^2) \\ = 0.168$$

- For the “purple” node

$$p_{i1} = (0/46); p_{i2} = (1/46); p_{i3} = (45/46)$$

$$G_i = 1 - ((0/46)^2 + (1/46)^2 + (45/46)^2) \\ = 0.043$$



NODE IMPURITY MEASURES (CLASSIFICATION)

- Entropy : $H_i = - \sum_{k=1}^K p_{ik} \log_2(p_{ik})$

- For the “green” node

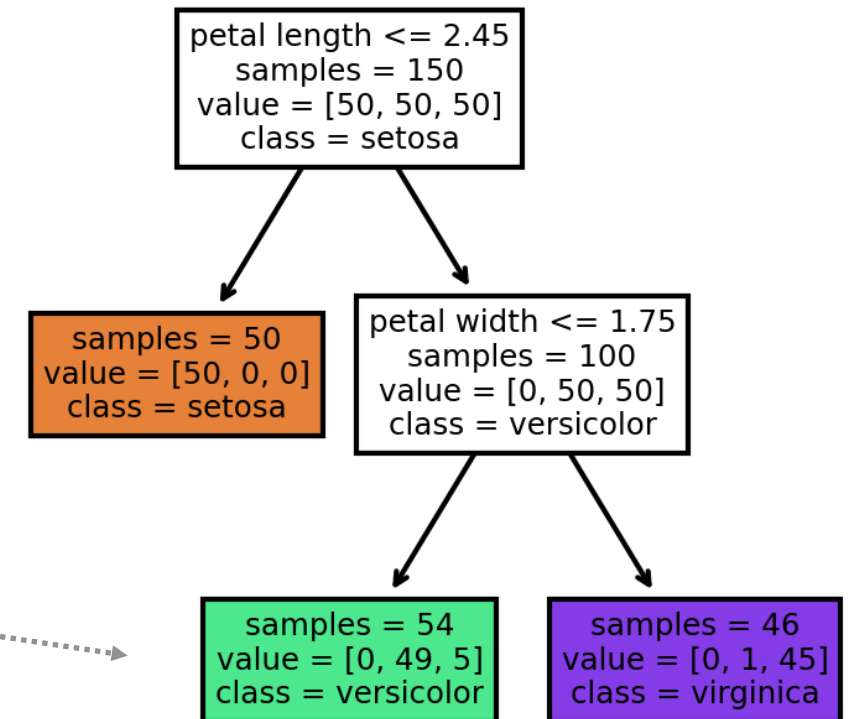
$$p_{i1} = (0/54); p_{i2} = (49/54); p_{i3} = (5/54)$$

$$H_i = -((49/54)\log_2(49/54) + (5/54)\log_2(5/54)) \\ = 0.445$$

- For the “purple” node

$$p_{i1} = (0/46); p_{i2} = (1/46); p_{i3} = (45/46)$$

$$H_i = -((1/46)\log_2(1/46) + (45/46)\log_2(45/46)) \\ = 0.151$$



GINI INDEX

- Ranges between 0 and $1 - \frac{1}{k}$ (where k is the number of classes)

(for binary classification, when $k = 2$, max value is 0.5)

- Gini index is 0 when all elements belong to the same class, indicating perfect purity
- Penalizes mixed nodes; favors splits that isolate dominant classes.
- It's maximum value is reached when the classes are distributed as equally possible

(for binary classification $G = 0.5$ indicates a perfect 50-50 split)

ENTROPY

- Ranges between 0 and $\log_2(k)$ (where k is the number of classes)

(for binary classification, when $k = 2$, max value is 1)

- Entropy is 0 when all elements belong to the same class, indicating perfect purity or “no disorder”
- It's maximum value is reached when the classes are distributed as equally possible

(for binary classification $H = 1$ indicates a perfect 50-50 split)

INFORMATION GAIN

- Entropy is used to calculate the **information gained** from the split
- Information gain:
 - Entropy(parent) – Entropy(split)

$$H(\text{parent}) = - \sum_{k=1}^K p_k \log p_k \quad (1)$$

$$H_{\text{split}} = \sum_{j=1}^J \frac{n_j}{n} H(\text{child}_j) \quad (2)$$

$$IG = H(\text{parent}) - H_{\text{split}} \quad (3)$$

GINI IMPURITY VS ENTROPY

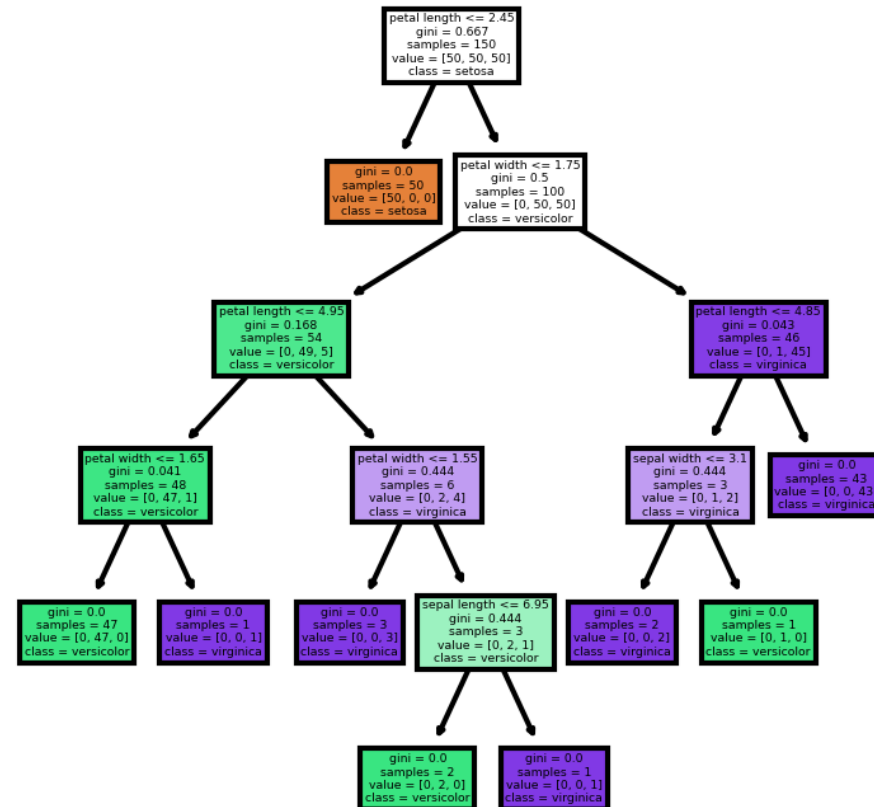
- The default in Scikit-Learn is Gini because it is faster to compute
- For decision tree construction, most of the time they lead to similar trees
- If they differ:
 - Entropy tends to produce more balanced trees
 - Gini Impurity tends to isolate the most frequent class onto its own branch
 - (See <https://hom1.info/19> for more details)

THE CART ALGORITHM

There are other tree algorithms that can produce nodes with more than two children

- **Binary**
 - Nodes can only split into two groups
 - **Top down**
 - Start at the top of the tree
 - **Efficient Split Finding**
 - Consider only the midpoint between consecutive unique values
 - **Greedy**
 - Finds the optimal split at the current level
-

Trees can grow very large if left unconstrained



REGULARIZATION

- Recall that regularization means constraining the model to prevent overfitting
- Decision Trees can be regularized by
 - Constraining growth (pre-pruning)
 - Post-pruning the unconstrained tree (cost complexity pruning)

PRE-PRUNING REGULARIZATION

- Stop tree growth before it perfectly classifies the training data
 - **max_depth**: maximum depth of tree
 - **min_samples_split**: minimum number of samples required to split an internal node.
 - **min_samples_leaf**: minimum number of sample required to be at a leaf node
 - **max_leaf_nodes**: maximum number of leaf nodes
 - **min_impurity_decrease**: a node will be split if the split induces a decrease of impurity greater than or equal to this value

reduce “max” parameters and/or increase “min” parameters if tree is overfitting

POST-PRUNING REGULARIZATION

- Cost complexity pruning (CCP)
 - Remove nodes that do not significantly reduce impurity
 - Prune nodes from bottom up if it results in a net improvement in the cost-complexity measure
$$R_{\alpha}(T) = R(T) + \alpha \tilde{|T|} \text{ where}$$
 - $\tilde{|T|}$ is the number of terminal nodes and
 - $R(T)$ is the total impurity (Gini, Entropy, MSE)
 - $\alpha \geq 0$ is the regularization parameter controlling the trade-off- CCP finds the subtree that minimizes $R_{\alpha}(T)$
- **ccp_alpha**: subtree with largest $R_{\alpha}(T)$ that is smaller than `ccp_alpha` will be chosen

COST-COMPLEXITY PRUNING (CCP): INTUITION

Minimize $R_\alpha(T) = R(T) + \alpha|T|$

- Why does increasing α make trees smaller?
 - α multiplies the number of leaves.
 - Larger $\alpha \rightarrow$ larger penalty for each additional leaf.
 - A split must reduce impurity **enough** to justify its complexity cost.
 - If impurity reduction is small, the penalty dominates $\rightarrow$ subtree is pruned.
-

PRE- VERSUS POST-PRUNING

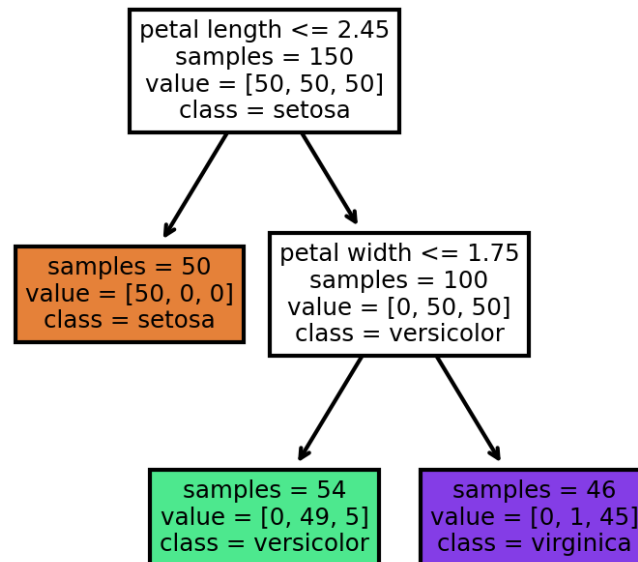
- Choice depends on:
 - Specific dataset and overfitting behavior observed
 - Domain knowledge
 - Model performance on a validation set
 - Personal preference

MAKING PREDICTIONS

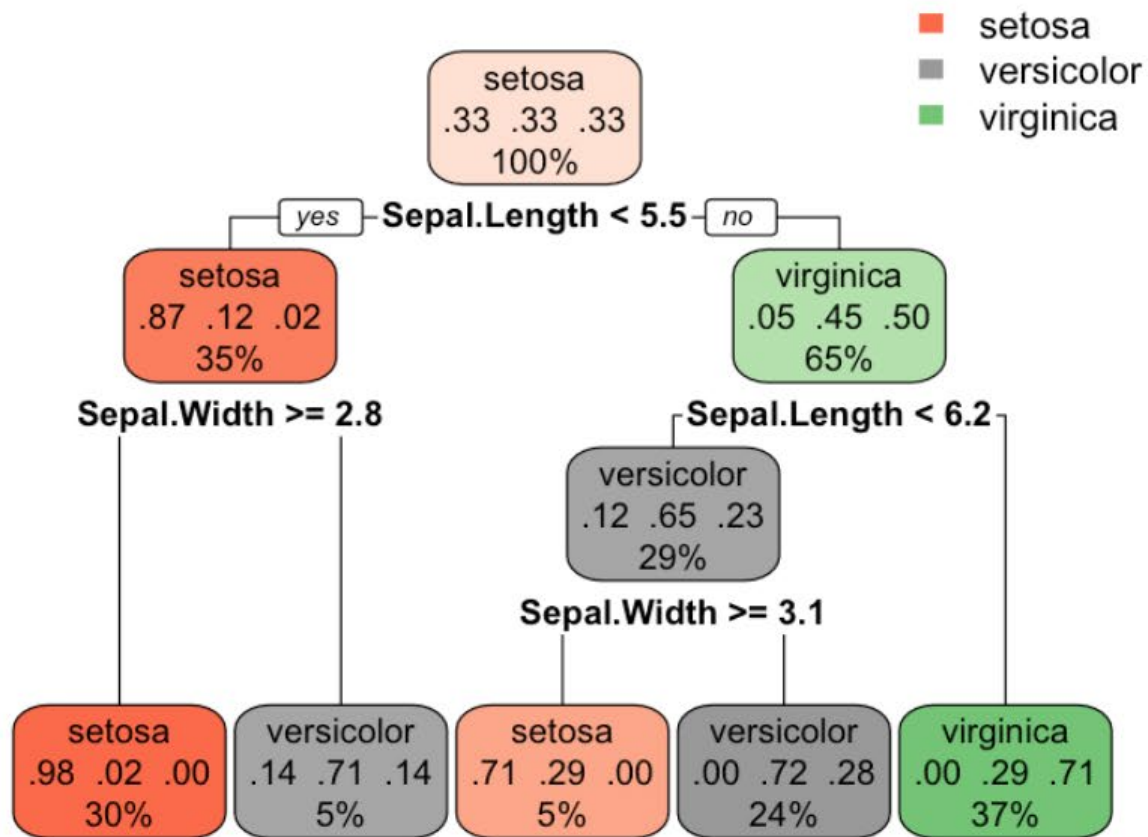
- We can predict the target of a new instance by following the tree from top to bottom
- At the terminal (leaf) nodes predict using:
 - Classification: The majority class in the node
 - Regression: The mean target value of the training values in the node

MAKING PREDICTIONS (CLASSIFICATION)

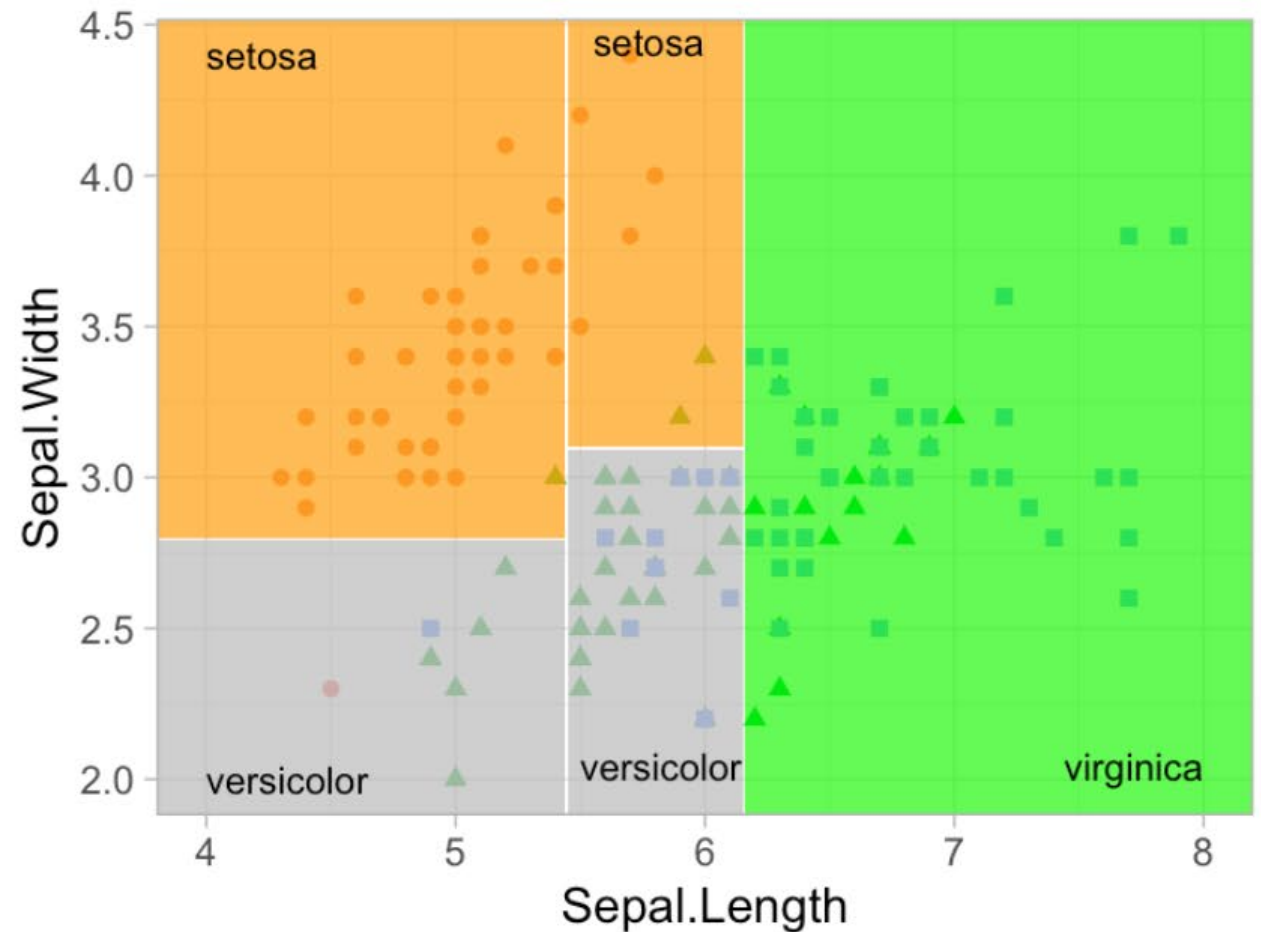
- We can predict the target of a new instance by following the tree from top to bottom
- At the terminal (leaf) nodes predict using:
 - Classification: The majority class in the node



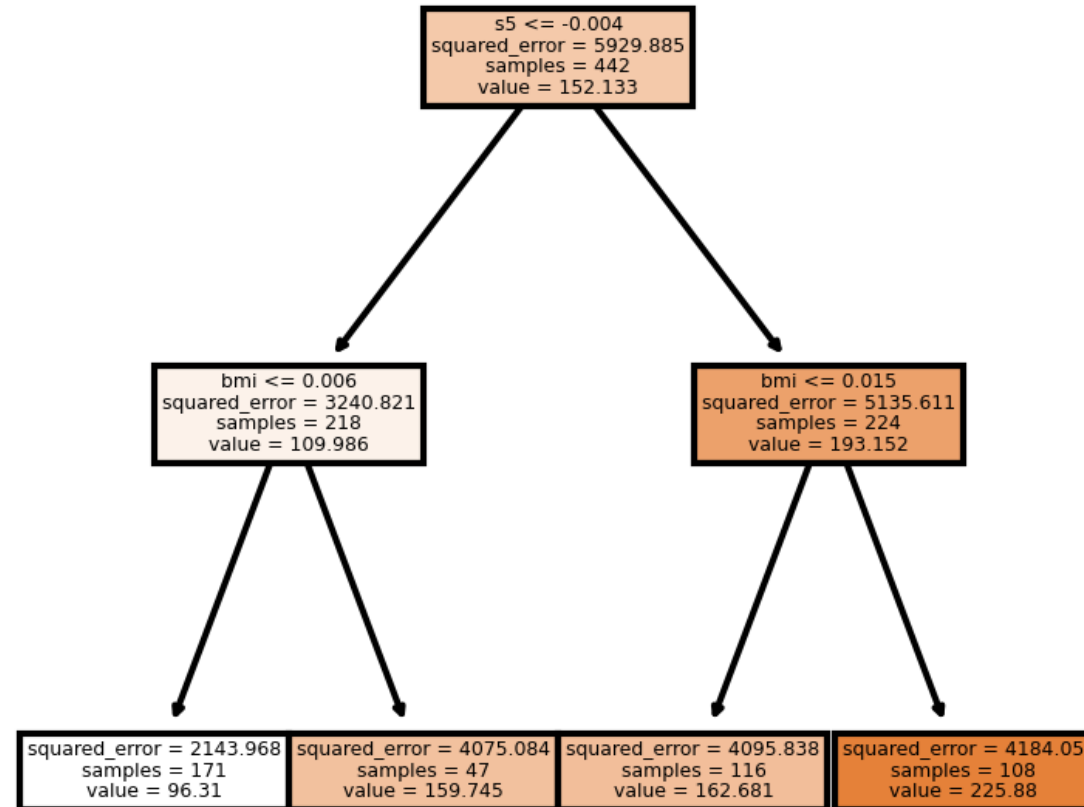
CLASSIFICATION BOUNDARY EXAMPLE



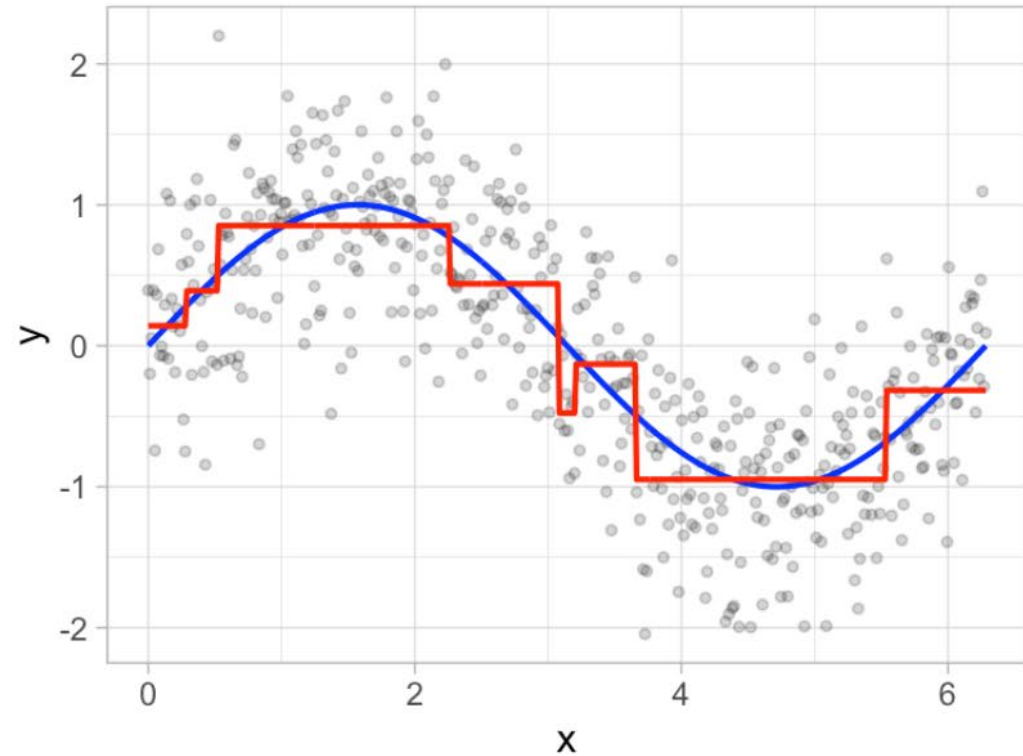
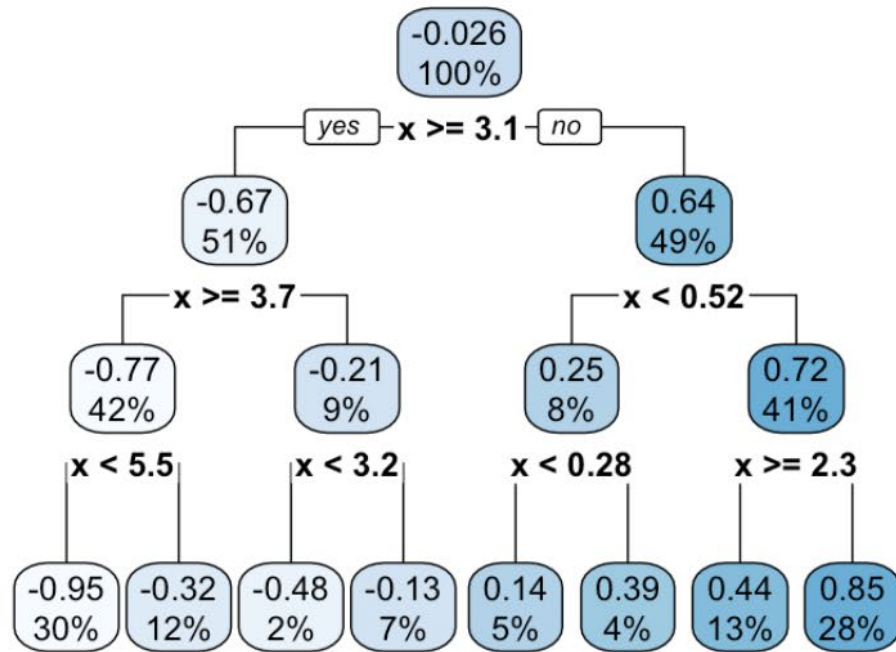
HOML R



MAKING PREDICTIONS (REGRESSION)



EXAMPLE: REGRESSION DECISIONS



EXAMPLE: REGRESSION DECISIONS

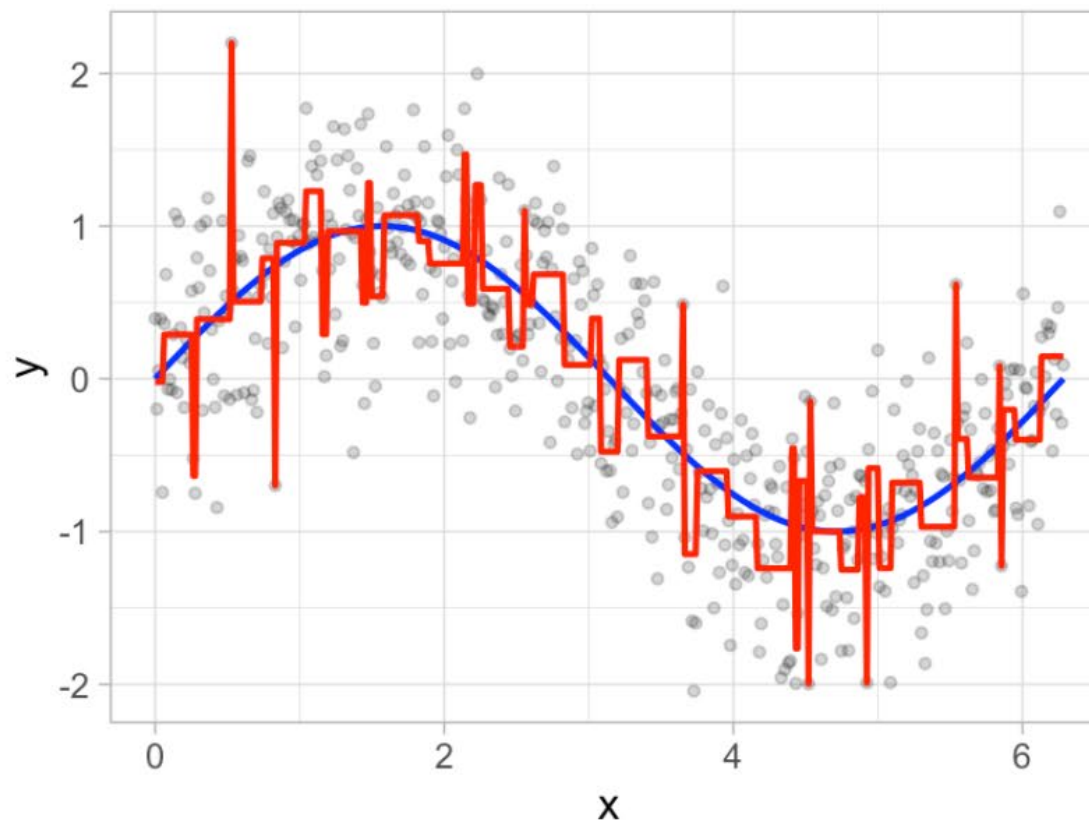
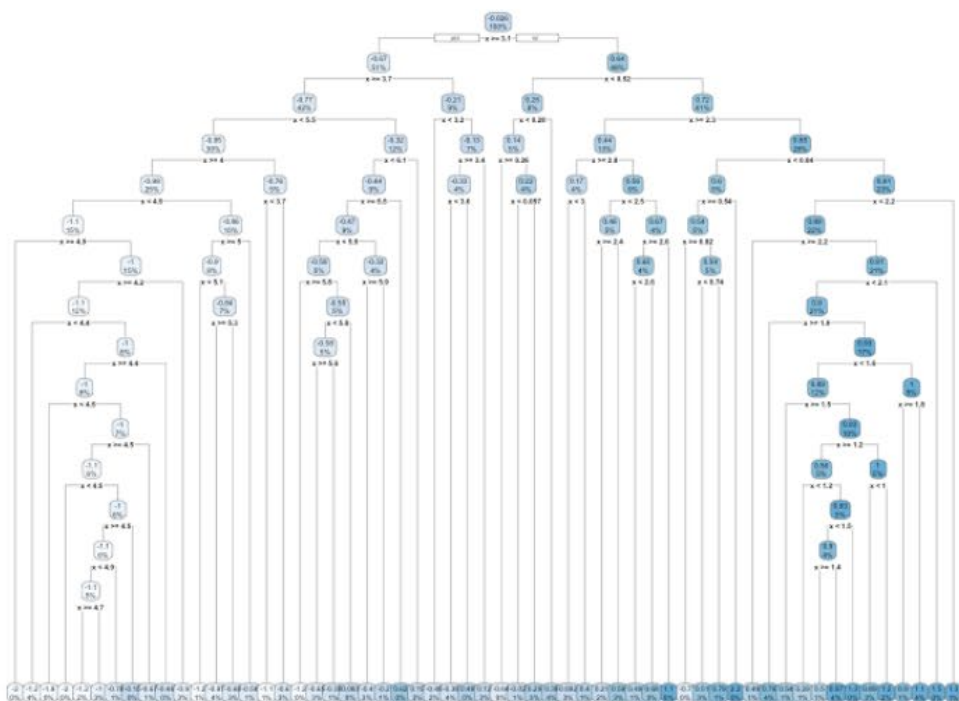
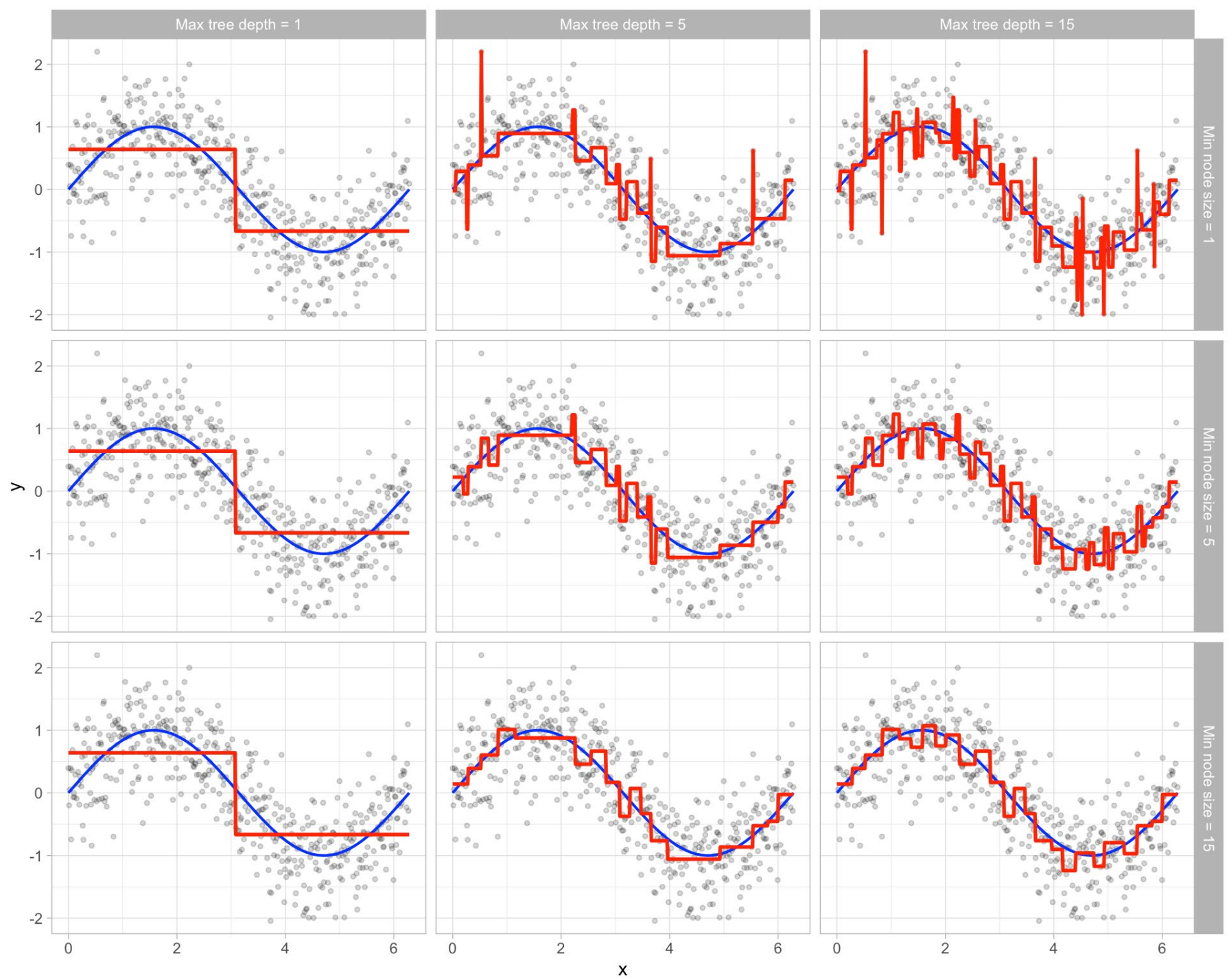


Figure 9.6: Overfit decision tree with 56 splits.



OTHER DECISION TREE ALGORITHMS

- CART is a very popular tree algorithm but it isn't the only one
(though it is the only one in scikit-learn)
 - Other algorithms:
 - **ID3** - designed for classification problems
 - Uses information gain to select the feature that best splits the set of items
 - Works only with categorical features
 - More than two branches can be created from a node
 - **C4.5** - extension of ID3 that handles both continuous and discrete features
 - **C5.0** - improved version of C4.5
 - **CHAID** - uses chi-square tests to determine the best split
-

PROS AND CONS OF TREES

- Pros
 - Easy to build and explain
 - Non-parametric and can easily capture non-linear relationships
 - Very little data preprocessing is required
 - Cons
 - Tendency to overfit
 - Difficulty with linear relationships
 - *High variance models (training set selection has a huge impact on the resulting tree!)*
-